Topic 9

Task1

```
import Foundation

struct Student {
    let name: String
    let marks: Int
    let dept: String
}

let students = [
    Student(name: "Asha", marks: 82, dept: "IT"),
    Student(name: "Ravi", marks: 35, dept: "CSE"),
    Student(name: "Meena", marks: 91, dept: "IT"),
    Student(name: "Kiran", marks: 56, dept: "ECE"),
    Student(name: "Divya", marks: 67, dept: "CSE"),
    Student(name: "Arjun", marks: 40, dept: "IT")
]

let passedStudents = students.filter { student in
    student.marks >= 40
}

let names = students.map { $0.name }

let average = students.map(\.marks).reduce(0, +) / students.count

let rankedStudents = students.sorted { $0.marks > $1.marks }

let studentsByDepartment = Dictionary(grouping: students, by: \.dept)
```

```
let passedFullForm = students.filter { student in
    student.marks >= 40
}

let passedUsingDollarZero = students.filter {
    $0.marks >= 40
}

let passedUsingKeyPath = students.filter(\.passed)

print("Passed Students:")
passedStudents.forEach {
    print("\($0.name): \($0.marks)")
}

print("\nStudent Names:")
print(names)

print("\nClass Average:")
print(Double(students.map(\.marks).reduce(0, +)) / Double(students.count))

print("\nRanked Students:")
rankedStudents.forEach {
    print("\($0.name): \($0.marks)")
}

print("\nStudents by Department:")
for (dept, group) in studentsByDepartment {
    print("\(dept): \(group.map(\.name))")
}
```

Output

```
Ranked Students:
Meena: 91
Asha: 82
Divya: 67
Kiran: 56
Arjun: 40
Ravi: 35

Students by Department:
CSE: ["Ravi", "Divya"]
ECE: ["Kiran"]
IT: ["Asha", "Meena", "Arjun"]

Passed using Full Form:
["Asha", "Meena", "Kiran", "Divya", "Arjun"]

Passed using $0:
["Asha", "Meena", "Kiran", "Divya", "Arjun"]

Passed using Key Path:
["Asha", "Meena", "Kiran", "Divya", "Arjun"]
```

Task2

```
import Foundation

func myFilter(_ items: [Int], _ predicate: (Int) -> Bool) -> [Int] {
    var result: [Int] = []

    for item in items {
        if predicate(item) {
            result.append(item)
        }
```

```
    }

    return result
}

func applyTwice(_ value: Int, _ f: (Int) -> Int) -> Int {
    return f(f(value))
}

class Button {
    var handlers: [() -> Void] = []

    func onTap(_ handler: @escaping () -> Void) {
        handlers.append(handler)
    }

    func tap() {
        for handler in handlers {
            handler()
        }
    }
}

let numbers = [10, 15, 20, 25, 30]

let evenNumbers = myFilter(numbers) { number in
    number % 2 == 0
}

print("Even numbers:", evenNumbers)
```

```
let result = applyTwice(3) { number in
    number * 2
}

print("Apply twice result:", result)

let button = Button()

button.onTap {
    print("Button tapped!")
}

button.onTap {
    print("Second handler executed!")
}

print("Tapping button:")
button.tap()
```

Output

```
Even numbers: [10, 20, 30]
Apply twice result: 12
Tapping button:
Button tapped!
Second handler executed!
```

Task 3

```
func makeCounter() -> () -> Int {
    var count = 0
```

```
    return {
        count += 1
        return count
    }
}

let a = makeCounter()
let b = makeCounter()

print(a(), a(), b(), a())
```

Output

```
1 2 1 3
```